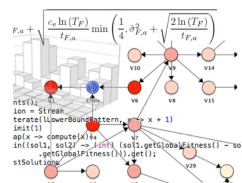


OPTIMISATION DISCRÈTE

Polytech Lyon - Filière informatique

Stéphane BONNEVAY

Polytech Lyon
Laboratoire ERIC
stephane.bonnevay@univ-lyon1.fr
me.theragoia.fr

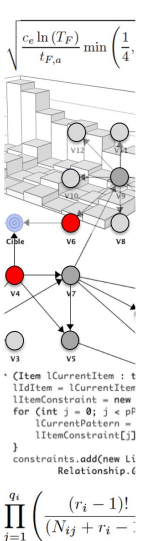


janvier 2024

1

Optimisation Discrète

Objectifs et compétences visées



L'**objectif** est de vous présenter des méthodes génériques de résolution de problèmes d'optimisation combinatoire, problèmes dont le nombre gigantesque de solutions potentielles rend impossible la mise en œuvre d'un algorithme permettant de trouver la solution optimale en temps raisonnable.

A l'issue de ce cours, vous :

- serez capable de modéliser un problème d'optimisation, puis d'adapter ou de créer un algorithme pour résoudre ce problème
- maîtriserez les principales métaheuristiques utilisées pour la résolution de problèmes d'optimisation combinatoire
- maîtriserez la résolution de programmes linéaires

Evaluation :

- Examen : le vendredi 7 juin à 9h45 → 11h15
- Projet à rendre avant le lundi 17 juin à 06h00

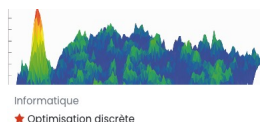
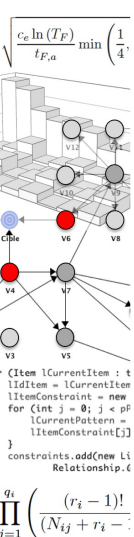
Stéphane BONNEVAY 2

2

Optimisation Discrète

Ressources pédagogiques

Les cours, TD, projets et données sont téléchargeables sur **MOODLE** :



Informatique
★ Optimisation discrète

Il existe de très nombreuses **ressources pédagogiques** sur Internet et des ouvrages à la bibliothèque, comme :

- « Métaheuristiques », P. Siarry, Eyrolles, 515 pages, 2014
- « Métaheuristiques – une introduction », B. Chopard et M. Tomassini, Presses Universitaires de Perpignan, 282 pages, 2017
- « Optimisation combinatoire – théorie et algorithmes », B. Korte et J. Vygen, éditions Lavoisier, 650 pages, 2017
- ...



Stéphane BONNEVAY 3

3

Optimisation Discrète

Temps d'exécution d'un algorithme

Soit un tableau $T[1, \dots, n]$ de dimension n contenant des valeurs entières prises aléatoirement dans l'intervalle discret $[-99, \dots, 99]$.

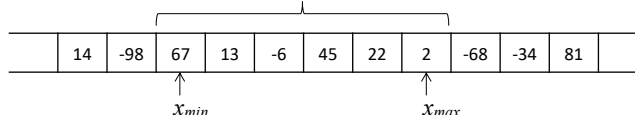
Problème :

Trouver deux entiers x_{min}, x_{max} ($1 \leq x_{min} \leq x_{max} \leq n$) tels que la quantité :

$$\sum_{i=x_{min}}^{x_{max}} T[i]$$

soit maximale (on note S_{max} la valeur maximum).

$$S_{max} = 143$$



Stéphane BONNEVAY 4

4

Optimisation Discrète

Temps d'exécution d'un algorithme

Algorithme « Trop Naïf » :

```

function TROPNAIF(T)
   $x_{min} = 1, x_{max} = 1, S_{max} = T[1]$ 
  for  $i = 1$  to  $n$  do
    for  $j = 1$  to  $n$  do
       $s = 0$ 
      for  $k = i$  to  $j$  do
         $s = s + T[k]$ 
      end for
      if  $s > S_{max}$  then
         $x_{min} = i, x_{max} = j, S_{max} = s$ 
      end if
    end for
  end for
  return  $x_{min}, x_{max}, S_{max}$ 
end function

```

 $O(n^3)$

	14	-98	67	13	-6	45	22	2	-68	-34	81	
--	----	-----	----	----	----	----	----	---	-----	-----	----	--

Stéphane BONNEVAY 5

5

Optimisation Discrète

Temps d'exécution d'un algorithme

Algorithme « Basic » :

```

function BASIC(T)
   $x_{min} = 1, x_{max} = 1, S_{max} = T[1]$ 
  for  $i = 1$  to  $n$  do
     $s = 0$ 
    for  $j = i$  to  $n$  do
       $s = s + T[j]$ 
      if  $s > S_{max}$  then
         $x_{min} = i, x_{max} = j, S_{max} = s$ 
      end if
    end for
  end for
  return  $x_{min}, x_{max}, S_{max}$ 
end function

```

 $O(n^2)$

	14	-98	67	13	-6	45	22	2	-68	-34	81	
--	----	-----	----	----	----	----	----	---	-----	-----	----	--

Stéphane BONNEVAY 6

6

Optimisation Discrète

Temps d'exécution d'un algorithme

Algorithme « Efficace » :

```

function EFFICACE(T)
    xmin = 1, xmax = 1, Smax = T[1]
    imin = 1, s1 = T[1], s2 = 0
    for i = 2 to n do
        s1 = s1 + T[i]
        if s1 - T[i] < s2 then
            imin = i, s2 = s1 - T[i]
        end if
        if s1 - s2 > Smax then
            xmin = imin, xmax = i, Smax = s1 - s2
        end if
    end for
    return xmin, xmax, Smax
end function

```

$$\sum_{i=x_{\min}}^{x_{\max}} T[i] = \sum_{i=1}^{x_{\max}} T[i] - \sum_{i=1}^{x_{\min}-1} T[i]$$

O(n)

	14	-98	67	13	-6	45	22	2	-68	-34	81	
--	----	-----	----	----	----	----	----	---	-----	-----	----	--

Stéphane BONNEVAY 7

7

Optimisation Discrète

Temps d'exécution d'un algorithme

Comparaison des temps de calcul (en millisecondes) :

Code en Java

```

lSol.mSMax = pTableau[0];
lSol.mXMin = 0;
lSol.mXMax = 0;
for (int i = 0; i < lNbElem; i++) {
    for (int j = i; j < lNbElem; j++) {
        lS = 0;
        for (int k = i; k <= j; k++) {
            lS += pTableau[k];
        }
        if (lS > lSol.mSMax) {
            lSol.mXMin = i;
            lSol.mXMax = j;
            lSol.mSMax = lS;
        }
    }
}
return lSol;

```

Algo « Trop Naïf »
O(n³)

```

lSol.mSMax = pTableau[0];
lSol.mXMin = 0;
lSol.mXMax = 0;
for (int i = 0; i < lNbElem; i++) {
    lS = 0;
    for (int j = i; j < lNbElem; j++) {
        lS += pTableau[j];
        if (lS > lSol.mSMax) {
            lSol.mXMin = i;
            lSol.mXMax = j;
            lSol.mSMax = lS;
        }
    }
}
return lSol;

```

Algo « Basic »
O(n²)

```

lSol.mSMax = pTableau[0];
lSol.mXMin = 0;
lSol.mXMax = 0;
lXMin = 0;
lS2 = 0;
lS1 = pTableau[0];
for (int i = 1; i < lNbElem; i++) {
    lS1 += pTableau[i];
    if ((lS1 - pTableau[i]) < lS2) {
        lXMin = i;
        lS2 = lS1 - pTableau[i];
    }
    if ((lS1 - lS2) > lSol.mSMax) {
        lSol.mXMin = lXMin;
        lSol.mXMax = i;
        lSol.mSMax = lS1 - lS2;
    }
}
return lSol;

```

Algo « Efficace »
O(n)

Stéphane BONNEVAY 8

8

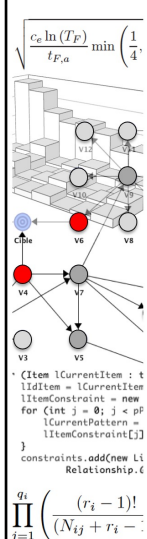
Optimisation Discrète

Temps d'exécution d'un algorithme

Comparaison des temps de calcul (en millisecondes) :

	Algo « Trop Naïf »	Algo « Basic »	Algo « Efficace »
	$O(n^3)$	$O(n^2)$	$O(n)$
n = 100	3	0	0
n = 1 000	69	4	0
n = 10 000	53 000	26	1
n = 100 000	51 589 948 (> à 14h)	2 329	3

Tests réalisés en novembre 2013 sur Intel Core i7 - 2.3 GHz



Stéphane BONNEVAY 9

9

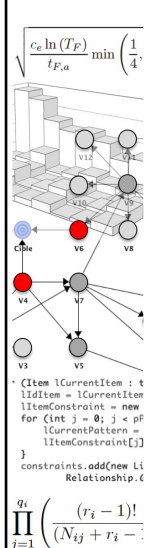
Optimisation Discrète

Temps d'exécution d'un algorithme

1 opération = 1 nanoseconde

n	$100n \log n$	$10n^2$	$n^{3.5}$	$n^{\log n}$	2^n	$n!$
10	3 μ s	1 μ s	3 μ s	2 μ s	1 μ s	4 ms
20	9 μ s	4 μ s	36 μ s	420 μ s	1 ms	76 années
30	15 μ s	9 μ s	148 μ s	20 ms	1 s	$8 \cdot 10^{15}$ a.
40	21 μ s	16 μ s	404 μ s	340 ms	1100 s	
50	28 μ s	25 μ s	884 μ s	4 s	13 jours	
60	35 μ s	36 μ s	2 ms	32 s	37 années	
80	50 μ s	64 μ s	5 ms	1075 s	$4 \cdot 10^7$ a.	
100	66 μ s	100 μ s	10 ms	5 heures	$4 \cdot 10^{13}$ a.	
200	153 μ s	400 μ s	113 ms	12 années		
500	448 μ s	2.5 ms	3 s	$5 \cdot 10^5$ a.		
1000	1 ms	10 ms	32 s	$3 \cdot 10^{13}$ a.		
10^4	13 ms	1 s	28 heures			
10^5	166 ms	100 s	10 années			
10^6	2 s	3 heures	3169 a.			
10^7	23 s	12 jours	10^7 a.			
10^8	266 s	3 années	$3 \cdot 10^{10}$ a.			
10^{10}	9 heures	$3 \cdot 10^4$ a.				
10^{12}	46 jours	$3 \cdot 10^8$ a.				

« Optimisation combinatoire – théorie et algorithmes. B.Korte et J.Vygen »

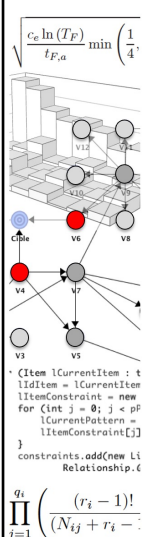


Stéphane BONNEVAY 10

10

Optimisation Discrète

Plan



1. Modélisation

Modélisation et proposition de résolution de problèmes classiques

2. Métaheuristiques

métaheuristiques à base de voisinage (descente, recuit simulé, méthode Tabou, ...) et de population (algorithmes évolutionnaires, colonies de fourmis, ...)

3. Programmation linéaire

programme linéaire, méthode de résolution graphique, méthode algébrique, méthode du simplexe, dualité, programmation linéaire en nombres entiers

4. Optimisation multiobjectif

dominance de Pareto, métaheuristiques

Stéphane BONNEVAY 11